Linear Algebra & Calculus for Deep Learning

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Build a concrete, geometric sense of what a “dimension” is — a feature vector vs. a multi-dimensional measurement
- Compute partial derivatives and assemble them into a gradient
- See the chain rule compose two functions — the same idea autograd automates, coming next lecture
- Use estimators (mean, std) to standardize features, and know what “estimator” means

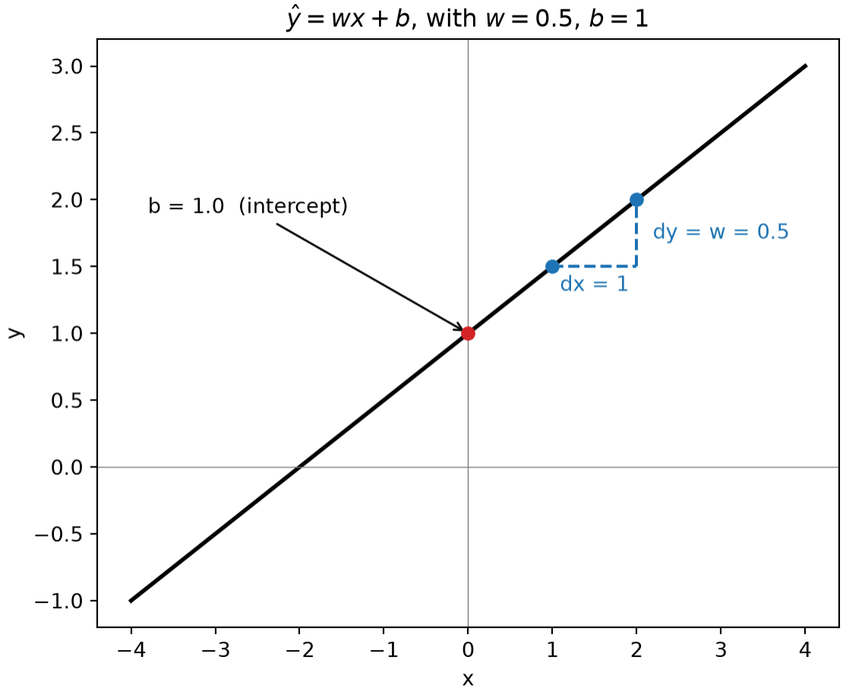
Optimizing by guessing

What is a model?

- A model is a function with **parameters** we can tune
- The simplest useful one is linear:

$$\hat{y} = wx + b$$

- `w` — slope: how much `ŷ` changes per unit of `x`
- `b` — intercept: where the line sits when `x = 0`
- This is the same `Ax + b` you've already seen — just one input, one output



Quick aside: this generalizes to vectors

- The example above used scalars — one x, one w — to keep things concrete
- In general, a model takes a whole feature vector **x** and a matching weight vector **w**:

$$\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$$

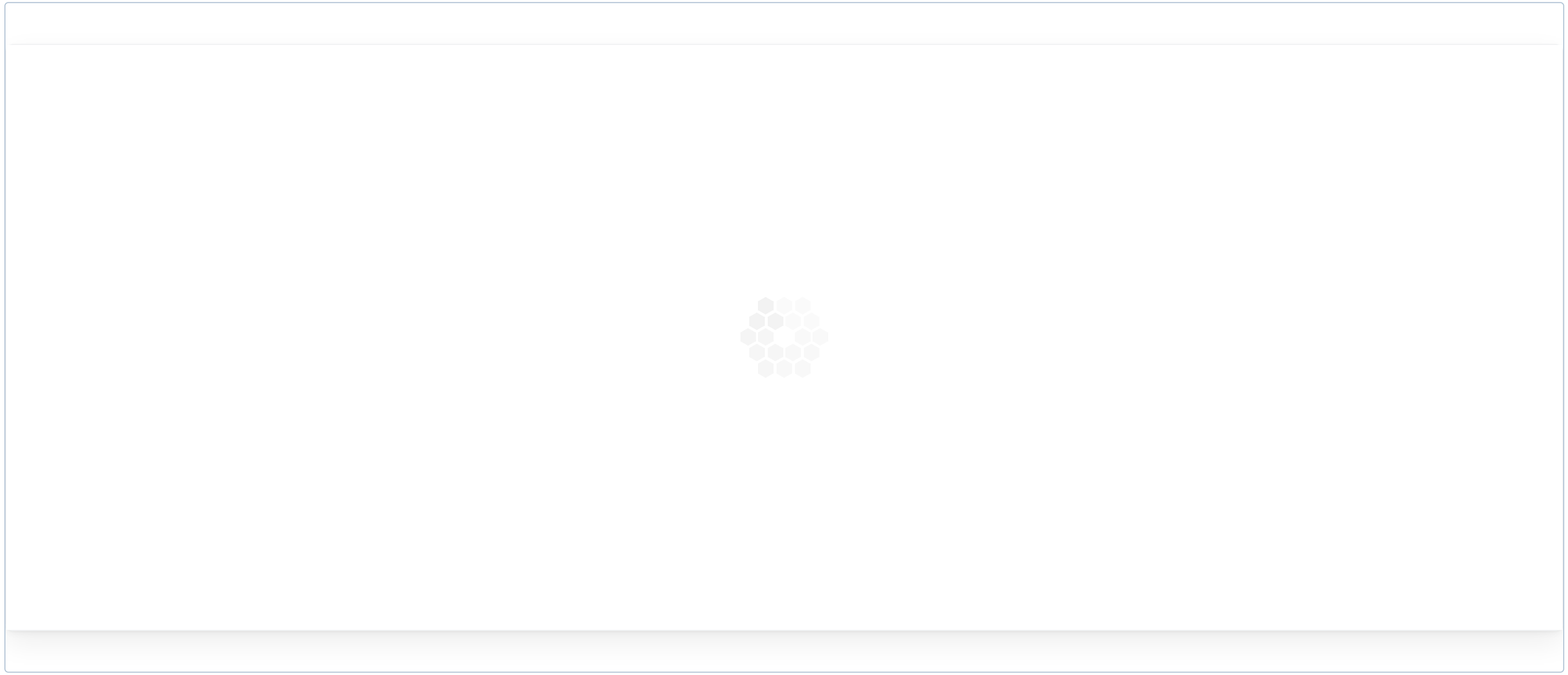
- Same idea, same shape (Ax + b) — just more numbers doing the same job
- We'll build up exactly what **w** and **x** mean, and what that · is doing, in a few slides

How do we improve a model?

- We have a way to score any choice of `(w, b)`: mean squared error
- So… how do we pick a *better* `(w, b)`?

Simplest possible idea: nudge them randomly, and see what happens.

Random search, live



This works… eventually

- Blue lines improved the fit, red lines made it worse — and by default, we take the step either way
- Check “only keep steps that improve” — about half the attempts still get thrown away (red), but it actually converges *faster*, because it never backslides
- Either way: there’s no sense of *direction*, only whether we got lucky
- With only two parameters this is already slow. Real models have millions.

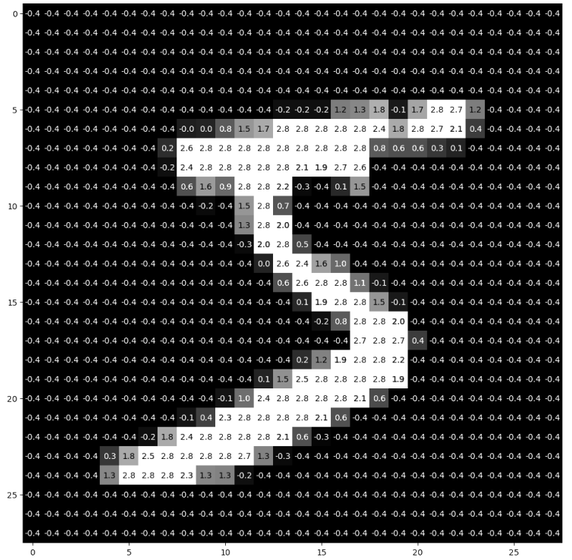
The question of the week: what if `choose_next_value()` didn’t have to guess?

Vectors, matrices, dimensions

What does “dimension” mean?

- Dimension means how many distinct input values we have
  - One house: `[sqft, bedrooms, age]`

- It can also describe the shape of our data
  - A greyscale image that is 28x28 would have shape `(1, 28, 28)`, so 784 input dimensions
  - It is also “2D” in the colloquial sense



> **Note**
>
> We can have both of these combined! For instance, `[sqft, bedrooms, age, (1, 28, 28)]`

A feature vector

```python
import torch

house = torch.tensor([1800.0, 3, 12])  # sqft, bedrooms, age
print(house.shape)
```

```
torch.Size([3])
```

- 3 numbers, 3 *different* things — shape `(3,)`
- Position matters because we've now chosen a convention for the data
  - In this dataset:
    - index 0 always means sqft
    - index 1 always means bedrooms
    - index 2 always means age

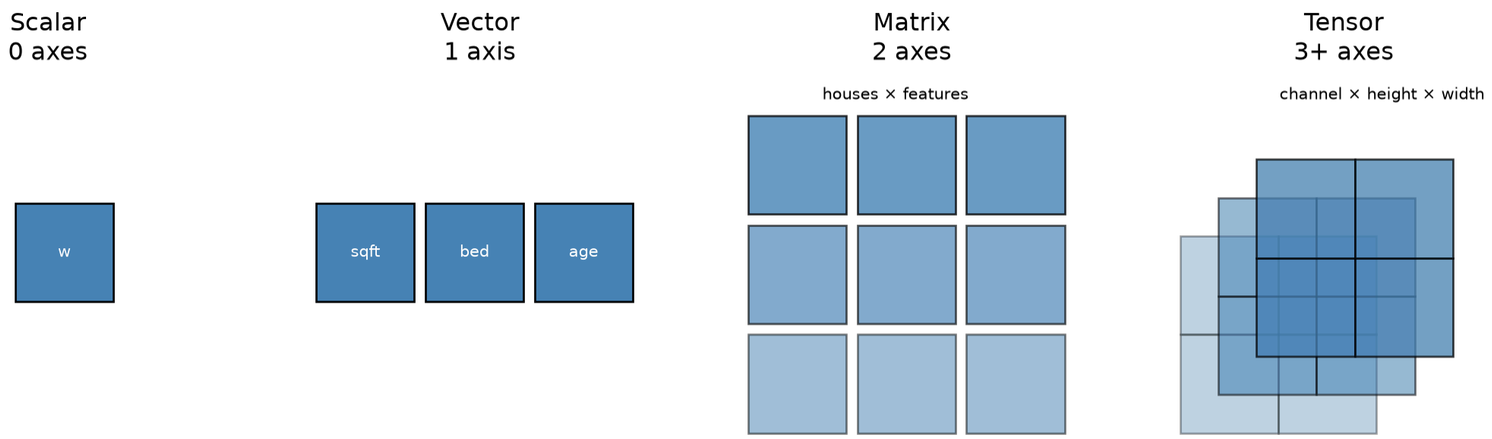
A multi-dimensional measurement

```
image = torch.zeros(1, 28, 28)  # channels, height, width
print(image.shape)
```

```
torch.Size([1, 28, 28])
```

- Unlike the feature vector, no axis here is a fundamentally different *kind* of thing
  - Axis 0 is “which channel,” axis 1 is “which row,” axis 2 is “which column” — all just *more pixels*
- `.shape` tells you the size of each axis, but not which axis is which — that’s a convention you have to know separately (here: channel, then height, then width)

Tensors: one word for all of these



- **Scalar** (0 axes) — a single number, like our `w`
- **Vector** (1 axis) — our feature vector `[sqft, bedrooms, age]`
- **Matrix** (2 axes) — a *batch* of feature vectors, one row per house
- **Tensor** (3+ axes) — our image: channel, height, width

- “Tensor” isn’t new math — it’s just the general word for “however many axes you need”
- Once you think in tensors, you stop asking “is this a vector or a matrix?” and just ask “what does each axis mean?”

Important Tensor Math

How big is a vector? Norms

- **L2 norm** — straight-line length:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- **L1 norm** — total distance if you could only move along axes:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|$$

```
x = torch.tensor([3.0, 4.0])
print(torch.linalg.norm(x, ord=2))
print(torch.linalg.norm(x, ord=1))
```

```
tensor(5.)
tensor(7.)
```

Does `||pred - actual||₂²` look familiar? That's exactly what mean squared error is measuring. More on this when we get to loss functions.

Why norms matter: measuring importance

- “Size” here rarely means literal length — it usually means **relative importance**
  - A large weight → that feature has a big influence on the prediction
  - A large gradient → that parameter is about to change a lot
  - A large error vector → the model is very wrong on this example
- We’ll spend real time controlling these sizes *on purpose*: regularization shrinks large weights, gradient clipping caps large gradients

Dot product: two ways to think about it

- **Algebraic:** sum of elementwise products

$$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$$

- **Geometric:** a measure of *alignment* — large when two vectors point the same way, near zero when they're perpendicular, negative when they point opposite ways

```
a = torch.tensor([1.0, 0.0])
b = torch.tensor([1.0, 0.0])
c = torch.tensor([0.0, 1.0])

print(torch.dot(a, b))  # same direction
print(torch.dot(a, c))  # perpendicular
```

```
tensor(1.)
tensor(0.)
```

- This dual meaning — "sum of products" *and* "similarity" — comes back constantly: cosine similarity, attention, and beyond

Why dot products matter: measuring agreement

- In plain terms, a dot product asks **“how much do these two things agree?”**
  - Pointing the same way → strong agreement (big positive number)
  - Perpendicular → no relationship (near zero)
  - Pointing opposite ways → active disagreement (big negative number)
- This is exactly how a model *evaluates* an input: each output is a dot product between the input and a learned weight vector — “how well does this input match what I’m looking for?”

Matrix-vector product: a batch of dot products

- Multiplying a matrix by a vector is just *many dot products done at once*, one per row

```
A = torch.tensor([[1.0, 0.0],
                  [0.0, 1.0],
                  [1.0, 1.0]])
x = torch.tensor([2.0, 3.0])

print(A @ x)
print(torch.dot(A[2], x))  # row 2 of A, on its own
```

```
tensor([2., 3., 5.])
tensor(5.)
```

- Row `i` of `A @ x` is exactly `A[i] · x` — nothing new, just done for every row together
- This is what a linear layer computes: one dot product per output neuron, in parallel

What this means: a matrix is a function

- $\mathbf{Ax}$ isn't just "multiply a grid by a list" — it's **evaluating $\mathbf{x}$** using $\mathbf{A}$
- Read it the same way you'd read $f(x)$: "put $\mathbf{x}$ into $\mathbf{A}$"
- Every row of $\mathbf{A}$ is a dot product "looking for" something — the matrix-vector product asks that "how much do you agree?" question for every row, all at once
- That's the entire forward pass of a linear layer, in one line:

$$\mathbf{Ax} + \mathbf{b}$$

Calculus: derivatives, partials, gradients

Derivatives: a fast recap

- The derivative of f at a point is its **slope** there — how fast f changes as x changes
  - Notation: $f'(x)$, or $\frac{df}{dx}$ – this latter one is much more explicit
  - $\frac{df}{dx}$ can be understood as “ratio of delta-f to delta-x” where “delta” is how we say “change in”

We only need a handlefule of rules:

Power Rule	$\frac{d}{dx}x^n = nx^{(n-1)}$
Constant Rule	$\frac{d}{dx}c = 0$
Constant Multiple Rule	$\frac{d}{dx}\big(c \cdot f(x)\big) = c \cdot f'(x)$
Chain Rule	$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$

The Chain Rule: Composing two functions

- $g(x) = 2x$
- $f(g) = g^2$
- Compose them: $f(g(x)) = (2x)^2 = 4x^2$

```
def g(x):
    return 2 * x


def f(g_val):
    return g_val ** 2


x = 3.0
print(f(g(x)))
```

```
36.0
```

Two ways to get the same derivative

- **Direct:** differentiate the composed function outright

$$\frac{d}{dx}\left(4x^2\right) = 8x$$

- **Chain rule:** differentiate each piece separately, then multiply

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx} = (2g) \cdot (2) = 4g = 8x$$

```python
x = 3.0
g_val = g(x)

df_dg = 2 * g_val     # derivative of f w.r.t. g, evaluated at g_val
dg_dx = 2.0           # derivative of g w.r.t. x

print(df_dg * dg_dx)
print(8 * x)  # direct derivative, for comparison
```

```
24.0
24.0
```

- Same answer, built from two *simple* pieces instead of one messier one
- This is *the* idea autograd is built on: break a big function into small pieces, differentiate each piece, multiply them together

Partial derivatives: one variable at a time

- Our function now takes more than one input:

$$f(x_1, x_2) = x_1^2 + 2x_2^2$$

- Notation: $\frac{\partial f}{\partial x_1}$, $\frac{\partial f}{\partial x_2}$

A **partial derivative** $\frac{\partial f}{\partial x_2}$ asks:

> "How does f change if I move *just* x_2, holding x_1 fixed?"

- $\frac{\partial f}{\partial x_1} = 2x_1$ — treat x_2 as a constant, differentiate like normal
- $\frac{\partial f}{\partial x_2} = 4x_2$ — same move, other variable

The gradient: a vector of partial derivatives

- Given the function:

$$f(x_1, x_2) = x_1^2 + 2x_2^2$$

- Stack the partials into a vector:

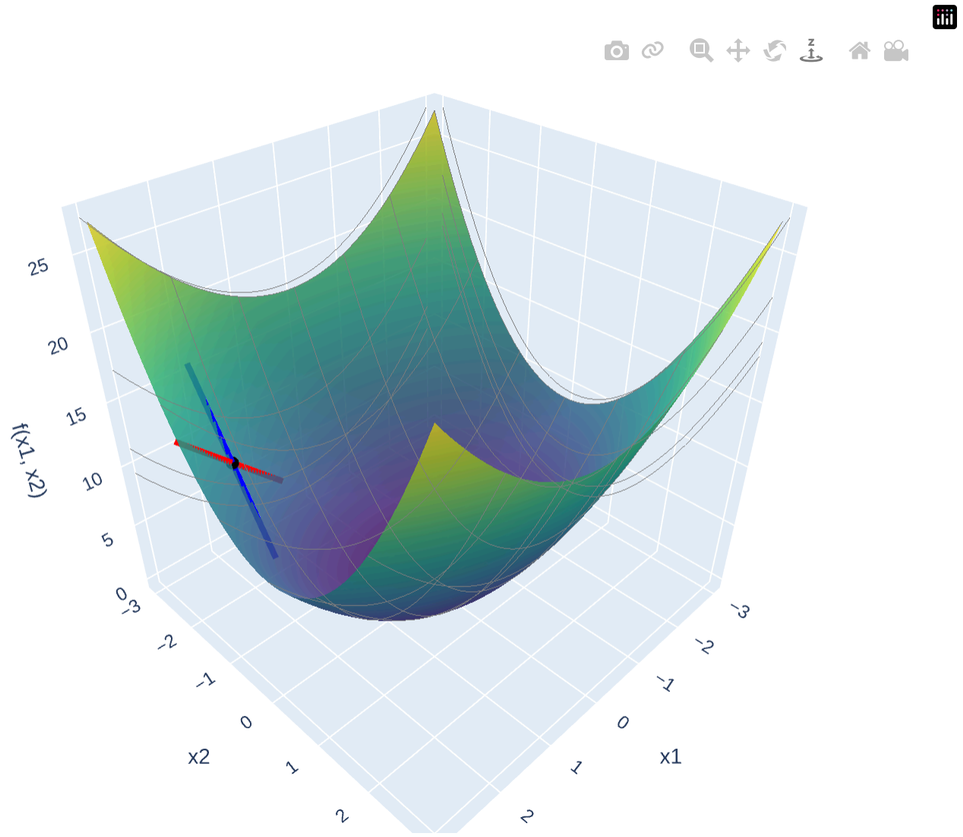
$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}\right] = [2x_1, 4x_2]$$

```
1 def grad_f(x1, x2):
2     return torch.tensor([2 * x1, 4 * x2])
3
4 print(grad_f(1.0, 1.5))
```

```
tensor([2., 6.])
```

- This vector points toward steepest *increase* — to go downhill, step in the *opposite* direction, $-\nabla f$
- Remember `choose_next_value()`? This is exactly the “smart” direction it was missing

What partial derivatives look like



- **Red line:** the slope along the x_1 direction, holding x_2 fixed — this is $\frac{\partial f}{\partial x_1}$
- **Blue line:** the slope along the x_2 direction, holding x_1 fixed — this is $\frac{\partial f}{\partial x_2}$
- The gradient just bundles these two slopes into a single vector
- Drag to rotate — get a feel for the bowl before we build on it

Try it yourself

Same plot, straight from the slide — paste this into your notebook to rotate it live, move the point, or change the function entirely:

```
import numpy as np
import plotly.graph_objects as go

x1 = np.linspace(-3, 3, 60)
x2 = np.linspace(-3, 3, 60)
X1, X2 = np.meshgrid(x1, x2)
F = X1 ** 2 + 2 * X2 ** 2

p1, p2 = 2.2, -1.8          # try moving this point around
fp = p1 ** 2 + 2 * p2 ** 2
t = np.linspace(-0.8, 0.8, 2)

d1 = 2 * p1  # df/dx1 at (p1, p2)
d2 = 4 * p2  # df/dx2 at (p1, p2)

fig = go.Figure()
fig.add_trace(go.Surface(
```

Stats: estimators

What is an “estimator”?

- An **estimator** is just a formula that turns data into a summary
  - Mean: `sum(data) / n` — estimates “the center”
  - Standard deviation — estimates “how spread out”
- Nothing exotic here — you’ve been using estimators since intro stats, just not the word

```
1 data = torch.tensor([2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0])
2
3 mean = data.mean()
4 std = data.std()
5
6 print(mean, std)
```

```
tensor(5.) tensor(2.1381)
```

- `data.mean()` and `data.std()` are estimators, applied to this particular sample
- A different sample gives a different estimate — that’s expected, it’s still doing its job

Standardization: putting estimators to work

- Raw features live on wildly different scales — `age` in years, `fare` in dollars
- **Standardization** rescales a feature using its own estimated mean and std:

$$x' = \frac{x - \hat{\mu}}{\hat{\sigma}}$$

- After standardizing, every feature has mean ≈ 0, std ≈ 1 — same footing, comparable scale

```
1 age = torch.tensor([22.0, 38.0, 26.0, 35.0, 28.0, 54.0])
2
3 age_mean = age.mean()
4 age_std = age.std()
5
6 age_standardized = (age - age_mean) / age_std
7 print(age_standardized)
```

```
tensor([-1.0293,  0.3624, -0.6814,  0.1015, -0.5074,  1.7542])
```

Remember the leakage rule from last time? Fit `mean`/`std` on the *training* split only, then reuse those exact numbers on validation/test.

So what is a model, statistically?

- Mean and std are estimators for *one number* — “what’s the center,” “what’s the spread”
- A machine learning model is an estimator too, but for an entire function relating inputs to outputs

$$\hat{y} = \text{model}(\mathbf{x})$$

- Same idea as `data.mean()`: fit some parameters to a sample, then use them to make a statement about data you haven’t seen yet
- The parameters *are* the model’s “mean” — a summary of the training data, generalized into a rule

The big picture about what a model is

That’s the statistical view. Zoomed all the way out, though, it’s simpler than that:

A model is a mapping from input data to output data

We need to specify clearly what our inputs and outputs are going to be

And be clever with how we think about them!

Wrap

Takeaways

- Vectors and matrices are how we represent “a measurement” and “many measurements” — tensors just generalize both
- Norms measure importance, dot products measure agreement, and Ax is evaluating a function — three ideas underneath every linear layer
- The gradient bundles partial derivatives into one vector that points uphill; the chain rule is what lets us compute the gradient of a composed function
- A model is a mapping from inputs to outputs — its parameters are just an estimator, fit on data, generalized into a rule

Coming up

- We still don’t have a *smart* `choose_next_value()` — just a random one
- Next lecture: gradient descent replaces the random guess with the gradient we just learned to compute
- And PyTorch’s autograd computes that gradient automatically, for functions far more complicated than today’s — no calculus by hand required